

Salesforce Developer Trainee

TECHNICAL EXAMINATION

Apex & Salesforce Development

Total Questions	Marks per Question	Total Marks
10	10	100

Candidate Name: _____ Trainee ID: _____ Date: _____

General Instructions

- Answer all 10 questions. Read each business scenario carefully before answering.
- Where Apex code is requested, write syntactically valid and logically complete code.
- Apply Salesforce security, bulk-processing, governor-limit, maintainability, and testability principles where relevant.
- Clearly state reasonable assumptions when a detail is not explicitly provided.
- Marks for each sub-question are shown separately. Each question carries 10 marks.
- Definition-only responses will receive limited credit unless they are applied to the given scenario.

Question 1

A sales organization stores customer information in Account, Contact, and Opportunity records.

The business wants a reusable Apex service that receives an Account Id and returns:

- Account Id and Name
- Related Contacts with Id, Name, Email, and Phone
- Related Opportunities with Id, Name, StageName, Amount, and CloseDate

The method will be called from multiple parts of the application. Users must only receive records they are permitted to access.

The organization also wants a separate search feature where a user enters text such as "ABC" and searches across both Account Name and Contact Name.

- Write the Apex class/method structure and the query required to retrieve the Account together with the required related Contact and Opportunity records. **(3 Marks)**
- Design the text-search operation across Account and Contact. Write the appropriate Apex query and explain why the selected query mechanism is suitable for this requirement. **(3 Marks)**
- Explain how the service should be designed so that record-level access and appropriate object/field access are respected when the method executes. **(2 Marks)**
- Assume the service may receive a list of multiple Account Ids instead of a single Account Id in the future. Explain how you would redesign the method to process the Accounts and their related records efficiently without placing SOQL queries inside loops. **(2 Marks)**

Total: 10 Marks

Question 2

A company automatically marks Accounts as Strategic when their total Opportunity value becomes greater than or equal to 5,000,000.

A developer implemented the following trigger:

```
trigger OpportunityTrigger on Opportunity (after insert, after update) {
    for (Opportunity opp : Trigger.new) {
        Account acc = [
            SELECT Id, Strategic__c
            FROM Account
            WHERE Id = :opp.AccountId
        ];

        List<Opportunity> opportunities = [
            SELECT Id, Amount
            FROM Opportunity
            WHERE AccountId = :opp.AccountId
        ];

        Decimal totalAmount = 0;
        for (Opportunity item : opportunities) {
            if (item.Amount != null) {
                totalAmount += item.Amount;
            }
        }

        if (totalAmount >= 5000000) {
            acc.Strategic__c = true;
            update acc;
        }
    }
}
```

The code works when users update one Opportunity at a time, but errors occur during data imports involving hundreds of Opportunities.

- Identify at least four technical problems or risks in the implementation above. **(3 Marks)**
- Explain how these problems can affect Salesforce governor limits, data processing, or transaction behavior during bulk operations. **(2 Marks)**
- Rewrite or redesign the implementation so that multiple Opportunities and Accounts can be processed safely in one transaction. **(3 Marks)**
- Explain how you would organize the trigger and supporting Apex classes so that business logic is not unnecessarily kept inside the trigger itself. **(2 Marks)**

Total: 10 Marks

Question 3

A company has the following Opportunity automation:

- A Validation Rule prevents an Opportunity from becoming Closed Won when Amount is blank or less than or equal to zero.
- A before-save Flow sets Business_Category__c.
- A before update Apex trigger calculates Risk_Level__c.
- An after update Apex trigger creates an Opportunity_Stage_Audit__c record when Stage changes.
- Additional record automation may update another Opportunity field after the initial save.

Users report that, in some transactions, the Opportunity trigger executes more than once and duplicate audit records are created.

- Write the Validation Rule formula that prevents an Opportunity from being saved as Closed Won when Amount is blank or not greater than zero. **(2 Marks)**
- Describe the relevant sequence of processing for the automation listed in the scenario, including the Validation Rule, before-save automation, before trigger, after trigger, and later record automation. **(3 Marks)**
- Explain why an Apex trigger may execute more than once during what the user considers a single save operation, and describe the possible impact on the audit-record creation logic. **(2 Marks)**
- Design an approach that prevents duplicate audit creation while still correctly detecting genuine Stage changes using the available trigger context data. **(3 Marks)**

Total: 10 Marks

Question 4

An organization has created a custom object called Customer_Assessment__c.

A developer writes an Apex class that:

- Retrieves Customer Assessment records.
- Reads the fields Risk_Score__c and Internal_Comment__c.
- Updates Approval_Status__c.

The class is called by users with different profiles and permission sets. During a security review, it is discovered that some users can execute the Apex code even though they should not be able to view certain assessment records or access Internal_Comment__c.

- Explain the difference between record-level access and object/field-level access in the context of this scenario. **(2 Marks)**
- Show how you would design the Apex class declaration and data-access operations so that the user's record-level access is appropriately considered. **(2 Marks)**
- Describe how CRUD and field-level security should be enforced when querying or updating the fields used by this service. Include appropriate modern Apex security mechanisms where relevant. **(3 Marks)**
- Assume the class currently executes its query and DML operations without considering the running user's access. Identify the security risks this creates and describe how you would redesign the service securely. **(3 Marks)**

Total: 10 Marks

Question 5

A company imports Leads from several external partners. A nightly Apex process receives 500 Lead records and attempts to insert them into Salesforce.

Some records may fail because of validation rules, duplicate rules, missing required information, or other DML errors. The current implementation is:

```
try {
    insert leadsToInsert;
} catch (Exception e) {
    System.debug('Error');
}
```

Management does not want the entire import to fail because a small number of Leads contain invalid data. Failed records must be identified for further investigation, while valid records should still be processed when appropriate.

- Identify the weaknesses in the current error-handling implementation. **(2 Marks)**
- Design or write Apex logic that allows valid Lead records to succeed while capturing detailed information about records that fail. **(3 Marks)**
- Explain how meaningful exception handling should be structured when an operation may encounter DML errors. Include the role of try, catch, and finally where each is appropriate. **(2 Marks)**
- Describe what error information should be recorded or returned so that support teams can investigate individual failed records without relying only on debug logs. **(3 Marks)**

Total: 10 Marks

Question 6

An organization has approximately 2 million Account records. Every night, the system must evaluate eligible Accounts and update Customer_Tier__c according to the following rule:

- Annual Revenue $\geq 10,000,000$ → Enterprise
- Annual Revenue $\geq 1,000,000$ and $< 10,000,000$ → Medium
- Annual Revenue $< 1,000,000$ → Small

Only active customer Accounts should be processed. The solution must work reliably with large data volumes and must provide a summary after processing is completed.

- Design a Batch Apex class for this requirement and explain the responsibility of its start, execute, and finish methods. **(3 Marks)**
- Write the main Apex structure required to query eligible Accounts, calculate the Customer Tier, and perform the updates in bulk. **(3 Marks)**
- Explain how batch scope size can affect governor-limit usage, transaction size, and overall processing. State the factors you would consider when selecting a suitable scope. **(2 Marks)**
- The business wants the final summary to include the total number of Accounts processed and the total number that failed. Explain whether state must be preserved between batch executions and how you would approach this requirement. **(2 Marks)**

Total: 10 Marks

Question 7

A development team has created an Apex service and trigger with the following behavior:

- When an Opportunity changes to Closed Won, an Opportunity_Reward__c record is created.
- No Reward should be created for any other Stage.
- The implementation must support bulk Opportunity updates.
- A separate asynchronous process recalculates reward information after the records are created.

The team currently has a test class that creates one Opportunity and only verifies that the Apex code executes without throwing an exception.

- a) Design the test data setup required to test this implementation without depending on existing organization data. **(2 Marks)**
- b) Describe and write the key assertions for a positive test and a negative test. **(2 Marks)**
- c) Design a bulk test involving a significant number of Opportunities and explain what behaviors should be asserted to verify that the implementation is correctly bulkified. **(2 Marks)**
- d) Explain how Test.startTest() and Test.stopTest() should be used when testing the asynchronous part of this implementation. **(2 Marks)**
- e) Apart from code coverage percentage, identify what makes an Apex test class meaningful and reliable for production deployment. **(2 Marks)**

Total: 10 Marks

Question 8

A company needs to integrate Salesforce with an external currency service.

Each Opportunity contains:

- Amount — stored in BDT
- Expected_Currency_Type__c — for example USD, EUR, or JPY
- Expected_Currency_Amount__c — value to be calculated from the external exchange rate

The external service requires authentication. It returns JSON containing the exchange rate. API endpoint information and credentials must not be hardcoded in Apex.

The service may return:

- HTTP 200 with valid JSON
 - HTTP 400 for an invalid currency
 - HTTP 401 for authentication failure
 - HTTP 500 for a service-side problem
 - A timeout or callout exception
- a) Describe how you would configure and use a Named Credential for this integration and show how Apex would reference the configured endpoint when performing the callout. **(2 Marks)**
 - b) Write the main Apex logic required to build the HTTP request, perform the callout, process a successful response, deserialize the relevant JSON data, and obtain the exchange rate. **(3 Marks)**
 - c) Design the error-handling logic for unsuccessful HTTP responses, invalid response content, and callout exceptions. **(2 Marks)**
 - d) The business now wants to process approximately 500 Opportunities. Analyze why performing all callouts directly from one synchronous transaction or from a trigger could be problematic, and propose a scalable integration architecture. **(3 Marks)**

Total: 10 Marks

Question 9

A developer created the following method to update Contacts related to Accounts whose Industry is Technology:

```
public static void updateContacts(List<Account> accounts) {
    for (Account acc : accounts) {
        List<Contact> contacts = [
            SELECT Id, AccountId, Description
            FROM Contact
            WHERE AccountId = :acc.Id
        ];

        for (Contact con : contacts) {
            if (acc.Industry == 'Technology') {
                con.Description = 'Technology Customer';
                update con;
            }
        }
    }
}
```

The method is called after a bulk integration inserts or updates up to 200 Accounts in a single transaction.

- Identify all major scalability, governor-limit, and maintainability problems in this implementation. **(3 Marks)**
- Rewrite the method using appropriate collections, bulk SOQL, and bulk DML so that it can safely process the full Account list. **(3 Marks)**
- Assume only Contacts whose Description actually needs to change should be updated. Modify the design to prevent unnecessary DML operations and explain why this matters. **(2 Marks)**
- Describe two additional Apex best-practice improvements you would make if this logic were part of a larger enterprise application. **(2 Marks)**

Total: 10 Marks

Question 10

A security-management company uses Salesforce to track software products in a custom object called Software_Product__c. There are approximately 100,000 Software Product records.

Every night, Salesforce must:

- Identify Software Products that require a vulnerability check.
- Call an external vulnerability API using the product name and version.
- Parse the returned JSON response.
- Create new Vulnerability__c records when vulnerabilities are found.
- Create Software_Product_Vulnerability__c junction records between the Product and each Vulnerability.
- Avoid creating duplicate Vulnerability or junction records.
- Continue processing other Products when one product encounters an API or data error.
- Store sufficient information to investigate failed processing.
- Respect Salesforce security when the resulting data is later accessed through Apex services.
- Be deployable with meaningful automated tests, including tests for the external callout behavior.

A single Product may return many vulnerability records, so a nightly execution can result in a large number of Salesforce records being created.

- a) Design the overall Apex architecture for this requirement. Clearly identify how large-volume record selection, external callouts, database operations, and separation of responsibilities would be handled. **(3 Marks)**
- b) Explain how you would prevent duplicate Vulnerability__c and Software_Product_Vulnerability__c records while processing large amounts of data. Your design should consider efficient use of queries, collections, and DML operations. **(2 Marks)**
- c) Analyze the governor-limit and transaction risks involved when one Product can return a large number of vulnerabilities. Describe how you would prevent a single transaction from attempting to process an unsafe volume of callouts, queries, or DML rows. **(2 Marks)**
- d) Design the automated test strategy for the solution, including successful API responses, API failures, duplicate data, bulk processing, asynchronous execution, and HTTP callout mocking. **(3 Marks)**

Total: 10 Marks

END OF EXAMINATION

Total Questions: 10 • Marks per Question: 10 • Total Marks: 100